

AgentX — Desktop App & Agent Runtime

Technical documentation for the incoming engineer

Go 1.26

Wails v2

React + TS

Fantasy LLM SDK

ClawHub Skills

BSC Wallet

Repository: `github.com/Agentx-network/agentx` (the "OpenClaw" Go runtime + desktop GUI)

Version at handover: **v0.8.49** · Working branch: `fix/bug-fixes`

Scope: this repo only — *not* the separate `agentx-backend` website (see §12)

Contents

- | | |
|-------------------------------------|--|
| 1. Executive summary | 9. Config & workspace (data on disk) |
| 2. Technology stack | 10. Feature reference |
| 3. High-level architecture | 11. Work done during this handover |
| 4. The three runtimes | 12. Related projects & boundaries |
| 5. Repository structure | 13. Known issues, gotchas & watch-outs |
| 6. Core concepts | 14. Git, branches & release flow |
| 7. How & where it runs (deployment) | 15. Quick-reference cheat sheet |
| 8. Build & run locally | 16. Onboarding checklist |

1. Executive summary

AgentX is a lightweight, self-hosted **AI agent runtime written in Go**, plus a cross-platform **desktop GUI**. It lets a user run a personal AI agent that can chat, use tools (shell, file I/O, web search, image generation), learn hot-loadable "skills", schedule tasks/reminders, talk over messaging channels (Telegram, Discord, etc.), and manage a BNB Smart Chain (BSC) crypto wallet.

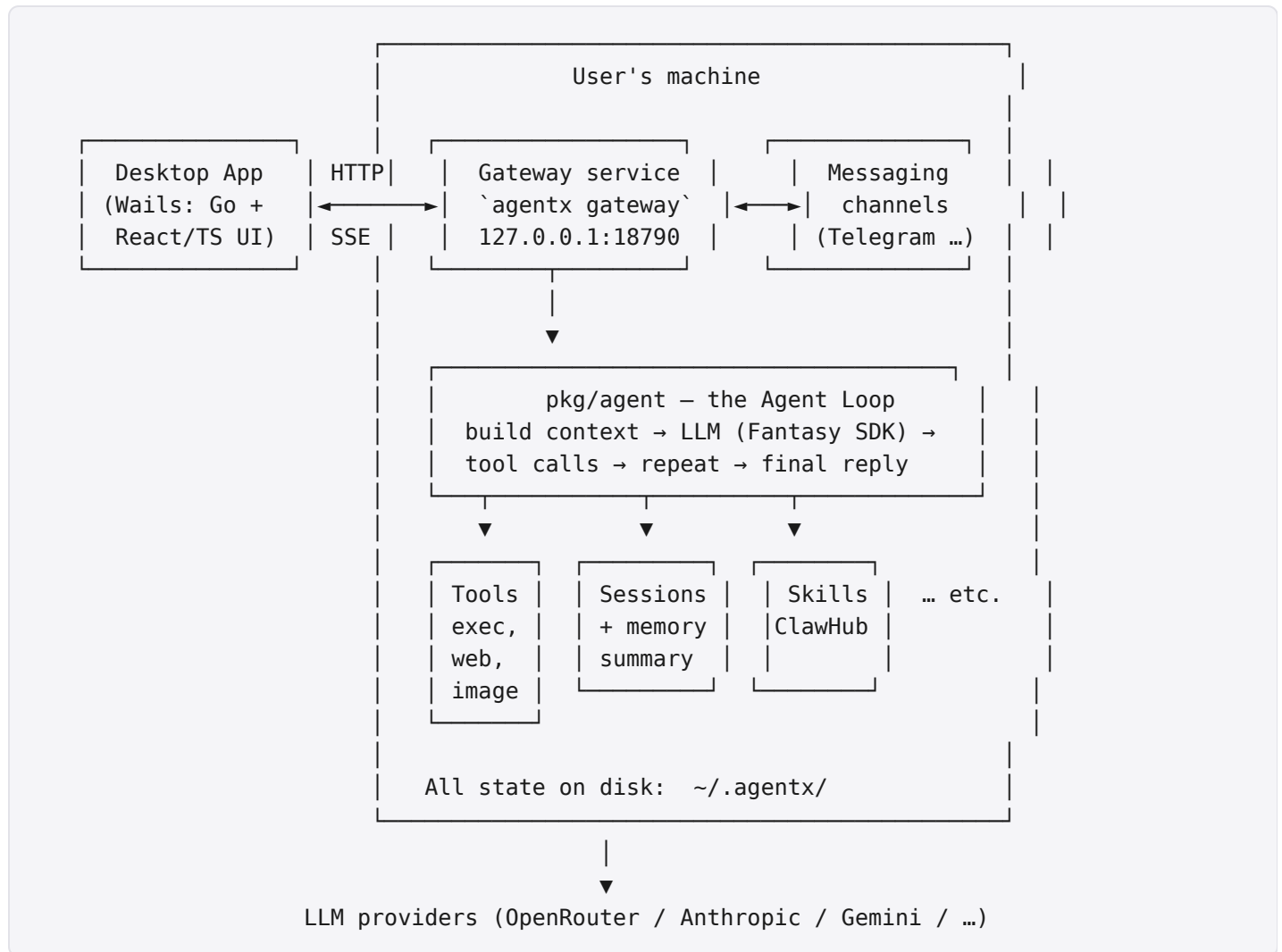
It is the client-side / runtime half of the broader **AgentX Network** product (a "decentralized AI agent economy"). This repository — internally nicknamed "**OpenClaw**" — contains the Go agent engine, the CLI, the background gateway service, and the Wails-based desktop application. The marketing site, the LaunchPad, and on-chain/economy pieces live in *other* repositories (see §12).

One-line mental model: A single Go binary (`agentx`) that runs an LLM tool-calling loop. It can run as a CLI, as a long-lived background **gateway** service, or be driven by a **desktop app**. All state lives in `~/ .agentx/` .

2. Technology stack

Layer	Technology	Notes
Core language	Go 1.26	Single static binary, ~5 MB RAM, CGO disabled for the CLI
LLM abstraction	Fantasy SDK <code>charm.land/fantasy v0.11.0</code>	Unified provider layer: Anthropic, OpenAI, Google/Gemini, OpenRouter, Groq, Cerebras, DeepSeek, Mistral, and more. Handles streaming, tool-calls, and multimodal (image/audio) input.
Desktop shell	Wails v2 <code>v2.11.0</code>	Go backend + web frontend in a native window (WebKit on Linux/macOS, WebView2 on Windows)
Desktop frontend	React + TypeScript + TailwindCSS + Vite	In <code>cmd/agentx-desktop/frontend/</code> . Talks to Go via auto-generated Wails bindings.
CLI framework	Cobra <code>spf13/cobra</code>	Subcommands: gateway, wallet, skills, cron, agent, auth, onboard, status, migrate, version, uninstall
Service manager	systemd (Linux), launchd (macOS), Task Scheduler (Windows)	Runs the gateway as a background service that auto-starts
Blockchain	BNB Smart Chain (BSC)	Wallet keygen + balances + token transfers via public JSON-RPC. ERC-8004 on-chain agent registry.
Skills	ClawHub marketplace (<code>clawhub.ai</code>)	Markdown-based "SKILL.md" skills, searched/installed at runtime
Messaging	Telegram, Discord, Slack, WhatsApp, LINE, WeCom, DingTalk, Feishu, QQ, OneBot	~10 channels via <code>pkg/channels</code>
Hardware I/O	I2C / SPI	For edge devices (e.g. Raspberry Pi, MaixCam)

3. High-level architecture



Key point: The desktop app is a thin client. It does not run the agent itself — it sends messages to the **gateway** over HTTP (`POST /api/chat` , Server-Sent-Events streaming). The gateway hosts the actual agent loop, tools, memory, and channels. This separation matters for deployment (see §7 and §13).

4. The three runtimes (one binary)

The same `agentx` binary behaves differently depending on how it is invoked:

Runtime	How it starts	Role
CLI	<code>agentx <command></code>	One-off commands: wallet ops, skill install, cron, onboarding, status. Also how you chat in a terminal.
Gateway	<code>agentx gateway</code> (run as a background service)	Long-lived process. Serves the HTTP API on <code>127.0.0.1:18790</code> , runs the agent loop, connects messaging channels, executes scheduled crons/heartbeats. This is the "backend" the desktop chat talks to.
Desktop	<code>agentx-desktop</code> (separate Wails binary)	Native GUI. Auto-starts the gateway if not running, then drives it over HTTP.

5. Repository structure

```
agentx/
├── cmd/
│   ├── agentx/                ← the CLI + gateway (single binary)
│   │   ├── main.go
│   │   └── internal/          ← cobra subcommands
│   │       ├── gateway/       ← HTTP API + agent wiring (helpers.go = /api/chat etc.)
│   │       ├── wallet/ skills/ cron/ agent/ auth/ onboard/
│   │       ├── status/ migrate/ uninstall/ version/
│   └── agentx-desktop/        ← the Wails desktop app
│       ├── *.go               ← Go services bound to the UI
│       ├── chat.go            ← chat + history + file attachments (SendMessage, PickAttachm
│       ├── agentsetup.go       ← skills search/install, bootstrap files
│       ├── dashboard.go        ← gateway start/stop/status
│       ├── service.go          ← installs the systemd/launchd service unit
│       ├── wallet.go configui.go schedulers.go installer*.go
│       └── frontend/          ← React + TS + Tailwind (Vite)
│           └── src/
│               ├── App.tsx      ← window.go bindings + routing + history load
│               ├── pages/ChatPage.tsx ← chat UI, attachments, markdown/image render
│               ├── pages/ConfigPage.tsx ← providers, models, skills search
│               └── pages/{Dashboard,Wallet,Schedulers,Installer}Page.tsx
├── pkg/                        ← the reusable engine (imported by cmd/*)
│   ├── agent/                 ← THE CORE. Agent loop, context building, tool orchestration,
│   │                           summarization, reminders. (loop.go, fantasy_runner.go, context.go, sur
│   ├── providers/             ← Fantasy SDK bridge + provider config + capability maps
│   ├── tools/                 ← agent tools: exec, read_file, write_file, list_dir, web_search,
│   │                           web_fetch, image_generate, find_skills, install_skill, cron, spawn, s
│   ├── session/               ← conversation persistence (sessions/*.json + *.transcript.jsonl)
│   ├── skills/                ← ClawHub registry client, skill loader, search cache
│   ├── channels/              ← Telegram, Discord, Slack, WhatsApp, LINE, WeCom, DingTalk, Feishu, QQ
│   ├── config/                ← config.json load/save + defaults
│   ├── wallet/                ← BSC keygen, balances, token transfers (tx.go, wallet.go)
│   ├── attach/                ← chat file-upload validation/storage (ADDED THIS HANDOVER)
│   ├── image/                 ← image generation providers (Gemini, OpenAI, Replicate, Seedance)
│   ├── cron/ heartbeat/       ← scheduled tasks + periodic self-checks
│   ├── bus/                   ← in-process message bus (inbound/outbound/stream deltas)
│   ├── routing/               ← maps a channel+peer to an agent + session key
│   ├── voice/                 ← audio transcription (used by channels)
│   ├── devices/ auth/ state/ migrate/ fileutil/ logger/ utils/ constants/ buildinfo,
├── Makefile                   ← build/install targets
├── install.sh / install.ps1
├── docker/                    ├── docs/                    └── config/
└── go.mod                     ← module github.com/Agentx-network/agentx, go 1.26
```

6. Core concepts

6.1 The agent loop (pkg/agent)

The heart of the system. For each user message:

1. **Build context** (`context.go`): system prompt (identity, rules, workspace info) + bootstrap files + installed skills summary + memory + session history/summary.
2. **Call the model** via the Fantasy SDK (`fantasy_runner.go` is the primary path; `runLLMIteration` is a legacy path). The model streams text and may emit tool calls.
3. **Execute tools** and feed results back; loop until the model produces a final answer or hits the iteration cap.
4. **Persist** the user + assistant messages, then maybe **summarize** (see 6.4).

`fantasy_runner.go` also contains resilience logic for weak models: recovering text-form tool calls, auto-running web search when the model "narrates" instead of acting, and (added this handover) auto-searching skills and attaching uploaded files.

6.2 Providers & models (pkg/providers)

Model selection is config-driven. The default is `Auto` (best available) , which resolves to `openrouter/auto` — OpenRouter's router that picks a model per request. Provider protocol (anthropic/openai/gemini/openrouter/...) is chosen in `fantasy_bridge.go` . Capability maps (`imagecapability.go` , `visioncapability.go`) declare which models can generate images or accept image/audio input.

6.3 Tools (pkg/tools)

Tool	What it does
<code>exec</code>	Run a shell command (e.g. ffmpeg, git). Powerful — the main way the agent "does" things.
<code>read_file</code> / <code>write_file</code> / <code>list_dir</code>	Sandboxed file I/O within the workspace (+ uploads dir).
<code>web_search</code> / <code>web_fetch</code>	Search the web / fetch a URL (Brave/Tavily/etc. per config).
<code>image_generate</code> / <code>configure_image_provider</code>	Create images (Gemini/OpenAI/Replicate/Seedance); manage the image provider + key.
<code>find_skills</code> / <code>install_skill</code>	Search ClawHub and install a skill into the workspace.
<code>cron</code>	Schedule reminders / recurring tasks.
<code>spawn</code> / <code>subagent</code>	Run background sub-agents for long tasks.
<code>i2c</code> / <code>spi</code>	Talk to hardware on edge devices.

6.4 Sessions & memory (pkg/session)

Every conversation is a "session", keyed by channel+peer (the desktop chat resolves to `agent:main:main`). Two on-disk representations, **kept separate on purpose**:

File	Purpose	Trimmed?
<code>sessions/<key>.json</code>	Agent context — what the model sees. Has <code>messages[]</code> + a text <code>summary</code> .	Yes — when a chat exceeds ~20 messages or ~75% of the context window, older messages are summarized into <code>summary</code> and dropped (<code>summarize.go</code> → <code>TruncateHistory</code>).
<code>sessions/<key>.transcript.jsonl</code>	UI display history — append-only, one JSON message per line. <i>(Added this handover.)</i>	No — never summarized. The desktop reads this so the chat window keeps full history even after the context is compacted.

Why two files? The model must stay within a token budget (so context is summarized), but a user expects their chat history to persist. The transcript decouples "what the agent remembers" from "what the UI shows." Before this, restarting the app made summarized-away messages disappear from the UI.

6.5 Skills (pkg/skills)

Skills are folders containing a `SKILL.md` (plus optional scripts) that teach the agent a capability. They live in `~/.agentx/workspace/skills/` . The agent can `find_skills` (search ClawHub) and `install_skill` (download+extract). A built-in **wallet** skill is auto-installed. Skill summaries are injected into the system prompt so the agent knows what it has.

6.6 Channels (pkg/channels)

Adapters that connect the agent to messaging platforms. Each channel receives messages, routes them (`pkg/routing`) to a session, runs the agent loop, and streams the reply back. Enabled/configured per-channel in `config.json` . Telegram is the one currently enabled in this environment.

7. How & where it runs (deployment)

7.1 On this machine (Linux)

Component	Where
Gateway service unit	<code>~/.config/systemd/user/agentx-gateway.service</code> (a systemd user service , auto-starts)
Gateway process	<code>agentx gateway</code> → listens on <code>127.0.0.1:18790</code>
Installed binary	<code>/usr/local/bin/agentx</code> (system-wide install) — root-owned
Desktop-installed binary	<code>~/.local/bin/agentx</code> (when installed via the desktop app)
Desktop app binary	<code>cmd/agentx-desktop/build/bin/agentx-desktop</code>
All user data	<code>~/.agentx/</code> (config, workspace, wallet, logs)

7.2 The gateway lifecycle (important)

The desktop app **auto-starts the gateway** on launch if it isn't already reachable (`main.go` → `DashboardService.StartGateway`). It locates the binary with `exec.LookPath("agentx")` , which finds whatever `agentx` is first on `PATH` (normally `/usr/local/bin/agentx`), and installs/refreshes the systemd unit to point there. The Dashboard has `RESTART` / `STOP` buttons that drive the same service.

7.3 Manual gateway control (Linux)

```
systemctl --user status agentx-gateway.service
systemctl --user restart agentx-gateway.service
systemctl --user stop agentx-gateway.service
tail -f ~/.agentx/gateway.log # runtime logs
```

8. Build & run locally

8.1 Prerequisites

Go 1.26+, Node.js + npm (for the desktop frontend), the Wails CLI (`go install github.com/wailsapp/wails/v2/cmd/wails@latest`), and on Linux the WebKit2GTK dev libs.

8.2 Build commands (Makefile)

```
# CLI + gateway binary → build/agentx-linux-amd64
make build

# Desktop app → cmd/agentx-desktop/build/bin/agentx-desktop
make desktop-build      # (wails build -tags webkit2_41)

# Desktop hot-reload dev mode
make desktop-dev

# Run the whole test suite
make test               # or: go test ./...
```

8.3 The redeploy loop (what you do after a backend change)

Critical: Chat/schedulers/wallet all run in the **gateway**, not the desktop app. Rebuilding only the desktop app will not pick up backend changes. You must rebuild the `agentx` binary, replace the one the service runs, and restart the gateway.

```
make build
systemctl --user stop agentx-gateway.service
cp build/agentx-linux-amd64 ~/.local/bin/agentx      # or /usr/local/bin (needs sudo)
systemctl --user start agentx-gateway.service
# then, for UI changes:
make desktop-build && ./cmd/agentx-desktop/build/bin/agentx-desktop
```

8.4 Launching the desktop app from a terminal (Linux gotcha)

If you launch from a shell running inside a **snap** (e.g. the VS Code snap terminal), the snap injects library paths that crash the WebKit window (`undefined symbol: __libc_pthread_init`). Launch with a clean environment instead:

```
env -i HOME=$HOME USER=$USER \
  PATH=/home/$USER/.local/bin:/usr/local/bin:/usr/bin:/bin \
  DISPLAY=:0 XAUTHORITY=/run/user/$(id -u)/gdm/Xauthority \
  DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/$(id -u)/bus \
  XDG_RUNTIME_DIR=/run/user/$(id -u) \
  ./cmd/agentx-desktop/build/bin/agentx-desktop
```

9. Config & workspace (data on disk)

9.1 ~/.agentx/config.json

The master config: providers & API keys, the model list, agent defaults (model, max tokens, temperature, max tool iterations, workspace path), channel settings, image providers, gateway host/port, and tool toggles. Edited via the desktop **Config** page or by hand. The default agent model is `Auto` (best available) = `openrouter/auto`.

9.2 ~/.agentx/workspace/

Path	What
AGENTS.md, SOUL.md, USER.md, IDENTITY.md	"Bootstrap" identity files — persona, rules, user profile, capabilities. Injected into the system prompt. Editable in the Config page.
HEARTBEAT.md	Instructions for the periodic heartbeat self-check.
sessions/	Per-conversation history — *.json (summarized) + *.transcript.jsonl (full display).
skills/	Installed skills (each a folder with SKILL.md).
uploads/	Chat file attachments (images/audio/docs/video). <i>(Added this handover.)</i>
memory/	Long-term memory notes (MEMORY.md + dated notes).
state/, cron/	Runtime state and scheduled jobs.

9.3 Wallet

~/.agentx/wallet.json (encrypted key) + tokens.json (tracked tokens). BSC only. Balance/tx via the public RPC <https://bsc-dataseed.binance.org/> (hardcoded in pkg/wallet/wallet.go — no Infura here; see §12).

10. Feature reference

Feature	Where	Notes
Chat (streaming)	Desktop Chat page ↔ gateway <code>/api/chat</code> (SSE)	Word-by-word streaming via the message bus
File upload	<code>pkg/attach</code> , <code>chat.go</code> , <code>ChatPage.tsx</code>	Docs → <code>read_file</code> ; images/audio → model vision/audio; video → <code>ffmpeg</code> via <code>exec</code>
Skills	Config page + <code>find_skills</code> / <code>install_skill</code>	ClawHub search + install; agent proactively offers a skill when it lacks a capability
Image generation	<code>pkg/image</code> , <code>image_generate</code> tool	Rendered inline in chat via an <code>IMAGE:<path></code> marker
Schedulers / reminders	Schedulers page, <code>pkg/cron</code> , <code>/api/schedulers</code>	"remind me in 10 min" → cron job; out-of-band escape for runaway crons
Wallet	Wallet page, <code>pkg/wallet</code> , <code>agentx wallet</code>	Generate/import/export, balances, token tracking (BSC)
On-chain registry	Dashboard "Register Agent"	ERC-8004 agent identity on BSC (metadata pinned to IPFS)
Channels	<code>pkg/channels</code>	Telegram/Discord/Slack/... — configure keys in Config

11. Work done during this handover

All on branch `fix/bug-fixes`. Confirm commit status with `git status` — some or all may still be uncommitted at handover.

11.1 Skill discovery fixes

- **Proactive skill offer:** when the model says it "can't do X", the runner auto-runs `find_skills` and offers to install a matching skill instead of dead-ending (`fantasy_runner.go` `shouldAutoSkillSearch` ; prompt rule in `context.go`).
- **Search recall:** the ClawHub search is prefix-matching, so "video compression" returned ~1 result but "video compress" returned ~9. Added query broadening (stopword-strip + stemming) in `pkg/skills/query_expand.go`, used by both the chat tool and the Config search.
- **UI:** clearing the Config search box now clears stale results.

11.2 Chat file upload (new feature)

- New `pkg/attach` package: classify (image/audio/doc/video), validate (type sniff, size caps, count/total limits), store into `workspace/uploads/`, and confine paths (security).
- Files flow through the existing `bus.InboundMessage.Media []string` (paths) contract. **Documents** → `read_file`; **images/audio** → sent to the model as Fantasy `FilePart`s (true vision/audio); **video** → path surfaced so the agent runs `ffmpeg` via `exec` (e.g. compression).
- New `pkg/providers/visioncapability.go` gates image/audio by model capability; if the model can't see/hear, it **accepts + warns** rather than failing.
- Desktop: paperclip picker (`PickAttachments`), attachment chips/thumbnails, inline render, and a marker block so attachments survive reload.

11.3 Full chat transcript (new)

- `SessionManager.AppendTranscript` writes an append-only `*.transcript.jsonl`; the desktop `GetChatHistory` prefers it (falling back to the summarized `.json`). Fixes chat history vanishing after restart once a conversation was summarized.

12. Related projects & boundaries

Read this to avoid the "Infura" confusion. There is a [separate](#) repository, `agentx-backend` (Python, MongoDB), which powers the **website / LaunchPad**. That is where the **Infura** (Ethereum RPC) integration lives — its key is stored in MongoDB (`agent_mcp` collection), not in env. **This repo (the Go desktop app/runtime) has no Infura and no LaunchPad code** — "LaunchPad" here is only a roadmap item in the README. If asked to "update the Infura key," it is almost certainly about the `agentx-backend` / deployed website, not this project.

Repo	What	Stack
Agentx-network/agentx (this)	Go agent runtime + CLI + gateway + desktop app	Go, Wails, React
agentx-backend	Website / LaunchPad / agent-creation backend	Python, MongoDB, MCP servers (incl. Infura Ethereum)

13. Known issues, gotchas & watch-outs

Gotcha	Detail / mitigation
Backend changes need the gateway restarted	Rebuilding the desktop app alone does nothing for chat/tools/memory. Rebuild the <code>agentx</code> binary and restart <code>agentx-gateway.service</code> (see §8.3).
Gateway binary "reverts"	The desktop app rewrites the service unit to <code>exec.LookPath("agentx") = /usr/local/bin/agentx</code> on launch. If you deploy a new build to <code>~/local/bin</code> but <code>/usr/local/bin</code> is on PATH, the desktop can point the service back at the old binary. Clean fix: install the new build to <code>/usr/local/bin/agentx</code> (<code>sudo</code>).
<code>openrouter/auto</code> credits	The configured OpenRouter key was out of credits (HTTP 402 in <code>gateway.log</code>). Chat/vision can fail intermittently. Fund the key or set a concrete model (e.g. a Gemini/GPT-4o key) in Config.
Model is "Auto"	Vision/audio capability can't be known ahead of time; the code optimistically attaches media for the <code>auto</code> router and warns for known text-only models.
Snap-shell launch crash	Launch the desktop app with a sanitized <code>env -i ...</code> (see §8.4) when starting from a snap terminal.
Old chat history not recoverable	The transcript feature records from its introduction onward; conversations summarized <i>before</i> it existed can't be restored.
Commit hygiene	Project preference: do <u>not</u> add "Generated with / Co-Authored-By" AI attribution to commits or PRs.

14. Git, branches & release flow

Remote	URL	Meaning
<code>origin / upstream</code>	<code>github.com/Agentx-network/agentx</code>	The main project
<code>myfork</code>	<code>github.com/AkshaySasi/agentx</code>	The outgoing engineer's fork (PRs originate here)

Working branch: `fix/bug-fixes`. Flow: branch off `main` → commit → push to the fork → open a PR into `Agentx-network/agentx:main`. (PR #17 from this fork was already merged; the tree of `origin/main` currently matches the last committed state here.)

Version bump: `make bump-version` updates the version; a `chore: bump version to x.y.z` commit + tag accompanies releases. Current: **v0.8.49**.

15. Quick-reference cheat sheet

PORT	127.0.0.1:18790	gateway HTTP API
CHAT API	POST /api/chat	{message, sessionKey, media[]} → SSE stream
OTHER APIs	/api/schedulers /api/notifications /api/reload /health /ready	
BINARY	/usr/local/bin/agentx or ~/.local/bin/agentx	
DESKTOP	cmd/agentx-desktop/build/bin/agentx-desktop	
SERVICE	systemctl --user {status restart stop} agentx-gateway.service	
LOGS	~/.agentx/gateway.log	
CONFIG	~/.agentx/config.json	
WORKSPACE	~/.agentx/workspace/ (bootstrap md, sessions/, skills/, uploads/, memory/)	
WALLET	~/.agentx/wallet.json	
BUILD	make build	→ build/agentx-linux-amd64
DESKTOP BUILD	make desktop-build	→ cmd/agentx-desktop/build/bin/agentx-desktop
TEST	go test ./...	
CORE CODE	pkg/agent/loop.go, fantasy_runner.go, context.go	(agent loop)
	pkg/tools/*	(agent tools)
	pkg/session/manager.go	(memory)
	cmd/agentx/internal/gateway/helpers.go	(HTTP API)
	cmd/agentx-desktop/chat.go + frontend/src/pages/ChatPage.tsx	(desktop chat)

16. Onboarding checklist for the next engineer

1. Install Go 1.26+, Node, the Wails CLI, and WebKit2GTK dev libs.
2. Clone `github.com/Agentx-network/agentx` ; check out `fix/bug-fixes` (or `main`).
3. `make build` and `make test` — confirm a green build.
4. Run `./build/agentx-linux-amd64 onboard` (or use the desktop Installer page) to create `~/.agentx/config.json` and set a provider API key.
5. `make desktop-build`, launch the desktop app (§8.4), and send a chat message — confirm the gateway auto-starts and streams a reply.
6. Read, in order: `pkg/agent/loop.go` → `fantasy_runner.go` → `context.go` → `cmd/agentx/internal/gateway/helpers.go`. That is the whole request path.
7. Internalise §13 (gotchas) — especially the gateway-restart and binary-reversion issues.
8. Skim §11 for the most recent changes and where they live.

AgentX Knowledge Transfer · prepared for the incoming engineer · repository github.com/Agentx-network/agentx · v0.8.49.

Verify anything security- or money-related (wallet keys, provider keys, on-chain actions) against the live code before acting.